⚡ DATABRICKS SPARK SQL CHEAT SHEET

Production Reference: DDL, DML, Liquid Clustering, Delta Lake ACID & Medallion Patterns

SPARK SQL | DBR 15.4+ LTS | PHOTON ENGINE | UNITY CATALOG

🏛️ Unity Catalog 3-Level Namespace & DDL

Governance

```sql
-- 3-Level Hierarchy: catalog.schema.table
CREATE CATALOG IF NOT EXISTS prod_lakehouse;
CREATE SCHEMA IF NOT EXISTS prod_lakehouse.silver_finance
  MANAGED LOCATION 'abfss://silver@storage.dfs.core.windows.net/';
USE CATALOG prod_lakehouse; USE SCHEMA silver_finance;

-- Liquid Clustering (Modern replacement for PARTITION BY)
CREATE TABLE transactions (
  txn_id BIGINT GENERATED ALWAYS AS IDENTITY,
  customer_id STRING,
  amount DECIMAL(18,2),
  txn_date DATE
) USING DELTA CLUSTER BY (txn_date, customer_id);

-- Zero-Copy Shallow Clone (instant pointer for dev/staging)
CREATE TABLE dev_txns SHALLOW CLONE transactions;
-- Deep Clone (full independent storage copy for DR/backup)
CREATE TABLE dr_backup DEEP CLONE transactions;
```

💡 **Liquid Clustering:** Avoids small-file skew, supports mutable keys, and re-clusters incrementally without full partition rewrites.

🔄 Delta Lake ACID Upsert (MERGE INTO)

DML / Medallion

```sql
MERGE INTO silver.customers AS target
USING bronze.stg_customers AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND source.is_deleted = true THEN
  DELETE
WHEN MATCHED AND target.hash_diff ≠ source.hash_diff THEN
  UPDATE SET
    target.email = source.email,
    target.status = source.status,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, status, created_at, updated_at)
  VALUES (source.customer_id, source.email, source.status,
          current_timestamp(), current_timestamp())
WHEN NOT MATCHED BY SOURCE AND target.status = 'ACTIVE' THEN
  UPDATE SET target.status = 'INACTIVE';
```

⚡ **Performance:** Always provide deterministic partition or cluster keys in the `ON` clause to enable Delta Dynamic File Pruning (DFP).

🛠️ Compaction, Z-Order & Vacuum

FinOps / Performance

```sql
-- Compact small files into ~1GB Parquet target files
OPTIMIZE silver.transactions;

-- Multidimensional Space-Filling Z-Order (Legacy/Static tables)
OPTIMIZE silver.transactions ZORDER BY (customer_id, txn_date);

-- Purge unreferenced historical parquet files older than 7 days
VACUUM silver.transactions RETAIN 168 HOURS;

-- Collect CBO statistics for adaptive join planning
ANALYZE TABLE silver.transactions COMPUTE STATISTICS FOR ALL COLUMNS;
```

⚠️ **Vacuum Caution:** Never set retention < 168 hours in production. Vacuuming 0 hours breaks concurrent active readers and invalidates time travel.

⏳ Delta Time Travel, Audit & Instant Rollback

Resilience

```sql
-- Query historical snapshot by version integer
SELECT * FROM silver.transactions VERSION AS OF 14;

-- Query snapshot by UTC timestamp string
SELECT * FROM silver.transactions
TIMESTAMP AS OF '2026-10-15 08:30:00';

-- Instant point-in-time recovery (Disaster Rollback)
RESTORE TABLE silver.transactions TO VERSION AS OF 13;

-- Inspect full commit history & user audit trail
DESCRIBE HISTORY silver.transactions LIMIT 10;
```

- `DESCRIBE HISTORY` records exact user, notebook ID, cluster ID, and metrics.
- `RESTORE TABLE` takes milliseconds — updates Delta transaction log pointer.

🔗 Chained CTEs, Window Slicing & Set Operations

Core SQL

```sql
WITH raw_txns AS (
  SELECT customer_id, region, amount,
         DENSE_RANK() OVER(PARTITION BY region ORDER BY amount DESC) AS rk
  FROM silver.transactions WHERE txn_date ≥ current_date() - 30
),
top_spenders AS (
  SELECT customer_id, region, amount FROM raw_txns WHERE rk ≤ 10
)
SELECT * FROM top_spenders
UNION ALL
SELECT customer_id, region, amount FROM silver.manual_overrides
EXCEPT DISTINCT
SELECT customer_id, region, amount FROM silver.fraud_blacklist;
```

💡 **Recursive CTE:** Use `WITH RECURSIVE` to traverse bill-of-materials or organizational hierarchies in a single SQL pass.

🥇 Medallion SQL & QUALIFY Deduplication

Architecture

```sql
-- Silver Cleanse: QUALIFY filter eliminates nested subqueries!
CREATE OR REPLACE TABLE silver.orders_cleansed AS
SELECT
  order_id, customer_id, amount,
  CAST(order_date AS DATE) AS order_date,
  _metadata.file_modification_time AS ingested_at
FROM bronze.orders_raw
WHERE amount > 0 AND order_id IS NOT NULL
QUALIFY ROW_NUMBER() OVER(
  PARTITION BY order_id ORDER BY ingested_at DESC
) = 1;

-- Gold Aggregation: Pre-computed dimension metrics
CREATE OR REPLACE TABLE gold.customer_metrics AS
SELECT customer_id, COUNT(*) AS orders, SUM(amount) AS ltv
FROM silver.orders_cleansed GROUP BY customer_id;
```

⚡ **QUALIFY Clause:** Replaces expensive `SELECT * FROM (SELECT ..., ROW_NUMBER() ...) WHERE rn=1` subqueries cleanly!

⚡ DATABRICKS SPARK SQL CHEAT SHEET

Analytical SQL, Complex Types, Higher-Order Functions, Pivot & Engine Tuning

ADVANCED SQL | ANALYTICAL FUNCTIONS | ROW FILTERS / MASKS

📊 Window Functions & Analytical Framing

Analytics

```sql
SELECT customer_id, txn_date, amount,
  -- Ranking: Row_Number (unique), Dense_Rank (no gaps)
  ROW_NUMBER() OVER(w_acct ORDER BY txn_date DESC) AS rn,
  DENSE_RANK() OVER(w_acct ORDER BY amount DESC) AS rk,
  -- Offsets: Prior & Next transaction amount
  LAG(amount, 1, 0.0) OVER(w_acct ORDER BY txn_date) AS prev_amt,
  LEAD(amount, 1, 0.0) OVER(w_acct ORDER BY txn_date) AS next_amt,
  -- Moving 7-Day Rolling Average (Physical rows)
  AVG(amount) OVER(w_acct ORDER BY txn_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS roll_7d_avg,
  -- Cumulative Year-to-Date Sum (Range)
  SUM(amount) OVER(w_acct ORDER BY txn_date
    RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS ytd_sum
FROM silver.transactions
WINDOW w_acct AS (PARTITION BY customer_id);
```

Pro Tip: If source data is clustered/sorted by the window partition key, Catalyst executes windowing without any network shuffle!

🧩 Complex Types: Structs, Arrays & JSON

Semi-Structured

```sql
-- Struct navigation, Array slicing, Map lookup
SELECT
  payload.user.id AS user_id,              -- struct field
  payload.tags[0] AS primary_tag,          -- array 0-index
  payload.attributes['tier'] AS tier,      -- map lookup
  -- Parse raw JSON string into typed Struct
  from_json(raw_payload, 'id BIGINT, name STRING, items ARRAY<STRING>') AS parsed,
  -- Convert Struct into JSON string
  to_json(named_struct('ts', current_timestamp(), 'code', 200)) AS json_str
FROM bronze.events;

-- Explode Array into relational rows
SELECT order_id, item.sku, item.qty, item.unit_price
FROM silver.orders
LATERAL VIEW EXPLODE(line_items) lv AS item;
```

- `LATERAL VIEW OUTER EXPLODE` preserves rows with empty/null arrays.
- `schema_of_json()` dynamically extracts schema from a sample JSON string.

⚡ Higher-Order Functions (In-Engine Array Ops)

Photon Vectorized

```sql
SELECT order_id,
  -- TRANSFORM: Map elements without exploding!
  TRANSFORM(line_items, x ->
    named_struct('sku', x.sku, 'gross', ROUND(x.price * 1.20, 2))
  ) AS taxed_items,

  -- FILTER: Retain only elements matching condition
  FILTER(line_items, x -> x.price > 100.0) AS luxury_items,

  -- EXISTS: Returns true if any element satisfies predicate
  EXISTS(line_items, x -> x.sku LIKE 'PROMO-%') AS has_promo,

  -- AGGREGATE: Reduce array to scalar sum
  AGGREGATE(line_items, 0.0, (acc, x) -> acc + x.price,
            acc -> ROUND(acc, 2)) AS total_order_sum
FROM silver.orders;
```

Architect Rule: Higher-Order functions run entirely in memory inside Tungsten/Photon C++ without row explosion or shuffle overhead.

🔒 Unity Catalog Row Filters & Column Masks

Security

```sql
-- 1. Column Masking (Mask card PAN unless in finance)
CREATE OR REPLACE FUNCTION mask_card_pan(pan STRING)
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('pci_auditors'), pan,
          CONCAT('****-****-****-', RIGHT(pan, 4)));

ALTER TABLE silver.payments
ALTER COLUMN card_pan SET MASK mask_card_pan;

-- 2. Row Filtering (Regional multi-tenant isolation)
CREATE OR REPLACE FUNCTION filter_by_region(region STRING)
RETURN IS_ACCOUNT_GROUP_MEMBER('global_admin') OR region = current_user();

ALTER TABLE silver.sales
SET ROW FILTER filter_by_region ON (region);
```

Fine-Grained Security: Applied transparently at storage read-time for BI, SQL, Python, and R without maintaining separate secure views.

🔄 PIVOT, UNPIVOT & Grouping Sets

Transformations

```sql
-- PIVOT: Rotate row values into columnar KPIs
SELECT * FROM (
  SELECT region, quarter, revenue FROM silver.sales
) PIVOT (
  SUM(revenue) FOR quarter IN ('Q1', 'Q2', 'Q3', 'Q4')
);

-- UNPIVOT: Rotate columnar metrics into tidy rows
SELECT * FROM silver.quarterly_kpis
UNPIVOT (revenue FOR qtr IN (q1, q2, q3, q4));

-- CUBE & ROLLUP: Multi-level executive aggregation
SELECT region, product, SUM(revenue) AS total
FROM silver.sales GROUP BY ROLLUP (region, product);
```

ROLLUP vs CUBE: `ROLLUP` produces hierarchical subtotals (Country > State > City); `CUBE` computes all 2^N combinations.

🚀 Join Strategy Hints & Session Performance Configuration

Engine Tuning

```sql
-- Optimizer Join Hints
SELECT /*+ BROADCAST(dim_cat) */
  f.txn_id, f.amount, dim_cat.category_name
FROM silver.fact_sales f
JOIN silver.dim_category dim_cat ON f.category_id = dim_cat.id;

-- Force Shuffle Hash Join
SELECT /*+ SHUFFLE_HASH(orders) */ *
FROM orders JOIN returns ON orders.id = returns.id;
```

Knob	Setting	Impact
`spark.sql.shuffle.partitions`	`auto`	Enables AQE dynamic partition sizing
`spark.sql.adaptive.enabled`	`true`	Runtime join re-planning & skew split
`spark.databricks.delta.optimizeWrite.enabled`	`true`	Pre-shuffle writes to prevent small files

DATABRICKS PYSPARK CHEAT SHEET

Production Reference: SparkSession, DataFrame API, Schemas, Joins, Windowing & Aggregations

PYSPARK | DATAFRAME API | DELTA LAKE 3.X | AUTO LOADER

SparkSession & Standard Imports

Setup

```python
from pyspark.sql import SparkSession, Window as W
from pyspark.sql import functions as F, types as T
from delta.tables import DeltaTable

# Pre-initialized in Databricks as 'spark'; Configured for Medallion
spark.conf.set("spark.sql.shuffle.partitions", "auto")
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")
spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true")
```

- `optimizeWrite` performs an exchange before writing to ensure 128MB-1GB files.
- `autoCompact` automatically checks and compacts files after individual write stages.

DataFrame Projections & Filtering

Core API

```python
# Projections, Expressions & Casts
df_clean = (df
    .select(
        "customer_id",
        F.col("amount").cast(T.DecimalType(18, 2)).alias("amount_eur"),
        F.to_date(F.col("created_at")).alias("txn_date")
    )
    .withColumn("tier",
        F.when(F.col("amount_eur") > 5000, "PLATINUM")
         .when(F.col("amount_eur") > 1000, "GOLD")
         .otherwise("STANDARD")
    )
    .filter((F.col("amount_eur") > 0) & F.col("customer_id").isNotNull())
    .dropDuplicates(["customer_id", "txn_date"])
    .orderBy(F.col("amount_eur").desc_nulls_last())
)
```

withColumns(): In Spark 3.3+, pass a dictionary to `df.withColumns(...)` to add multiple columns in a single projection pass!

Schema Definition (StructType & StructField)

Data Quality

```python
# Explicit schema avoids expensive inferSchema full-scans
txn_schema = T.StructType([
    T.StructField("txn_id", T.LongType(), nullable=False),
    T.StructField("cust_id", T.StringType(), nullable=False),
    T.StructField("amount", T.DecimalType(18, 2), nullable=False),
    T.StructField("attributes", T.MapType(T.StringType(), T.StringType())),
    T.StructField("items", T.ArrayType(T.StringType())),
    T.StructField("event_time", T.TimestampType())
])

# Read with enforced schema and corrupt record safety
df = (spark.read
      .schema(txn_schema)
      .option("mode", "PERMISSIVE")
      .option("columnNameOfCorruptRecord", "_corrupt")
      .json("abfss://landing@adls.dfs.core.windows.net/txns/"))
```

FinOps Sizing: Enforcing schemas on read eliminates a 2-pass scan over multi-terabyte JSON/CSV landing zones.

pyspark.sql.functions (F) Master Reference

Functions

Category	Top Functions
String	`F.concat_ws('_', c1, c2)`, `F.regexp_replace()`, `F.split()`, `F.trim()`
Date/Time	`F.to_date()`, `F.date_add(c, 30)`, `F.datediff(c1, c2)`, `F.date_trunc('MM')`
Nulls	`F.coalesce(c1, c2, F.lit('N/A'))`, `df.na.fill(0)`, `df.na.drop()`
Array	`F.explode()`, `F.array_contains()`, `F.size()`, `F.array_distinct()`
JSON	`F.from_json(col, schema)`, `F.to_json(struct)`, `F.get_json_object()`

Vectorization: Built-in `pyspark.sql.functions` execute natively in JVM/C++ Photon without Python serialization penalties!

Joins, Broadcast Hints & Skew Salting

Optimization

```python
# Broadcast Join: Eliminates shuffle for lookup tables (<100MB)
df_joined = df_fact.join(F.broadcast(df_dim), on="cat_id", how="left")

# Semi & Anti Joins (Replaces expensive IN / NOT IN subqueries)
df_active = df_users.join(df_purchases, on="user_id", how="left_semi")
df_churned = df_users.join(df_purchases, on="user_id", how="left_anti")

# Manual 2-Stage Skew Salting (When AQE cannot balance 100M+ key)
df_skewed = df_fact.withColumn("salt", (F.rand() * 10).cast("int"))
df_dim_exploded = (df_dim
    .withColumn("salt_arr", F.array([F.lit(i) for i in range(10)]))
    .withColumn("salt", F.explode("salt_arr")))
df_salted_join = df_skewed.join(df_dim_exploded, on=["key", "salt"])
```

Analytical Window Specifications (pyspark.sql.Window)

Analytics

```python
# Define Window Specifications
w_dept = W.partitionBy("dept_id").orderBy(F.col("salary").desc())
w_roll_7d = (W.partitionBy("account_id")
              .orderBy(F.col("txn_date").cast("timestamp").cast("long"))
              .rangeBetween(-7 * 86400, 0)) # 7 days in seconds

df_analyzed = (df
    .withColumn("dense_rank", F.dense_rank().over(w_dept))
    .withColumn("prev_salary", F.lag("salary", 1, 0).over(w_dept))
    .withColumn("salary_diff", F.col("salary") - F.col("prev_salary"))
    .withColumn("rolling_7d_total", F.sum("amount").over(w_roll_7d))
)
```

RangeBetween vs RowsBetween: `rangeBetween` handles calendar days and gaps; `rowsBetween` operates strictly on physical row indices.

DATABRICKS PYSPARK CHEAT SHEET

DeltaTable API, Auto Loader Streaming, Vectorized UDFs & Performance Tuning

LAKEHOUSE API | PYARROW VECTORIZED | STREAMING

DeltaTable Python API & Upserts

Lakehouse

```python
from delta.tables import DeltaTable

dt = DeltaTable.forName(spark, "prod_catalog.silver.customers")

# Pythonic ACID Merge / Upsert
(dt.alias("t")
   .merge(source_df.alias("s"), "t.cust_id = s.cust_id")
   .whenMatchedUpdate(
       condition="s.op = 'U'",
       set={"name": "s.name", "updated_at": "current_timestamp()"}
   )
   .whenMatchedDelete(condition="s.op = 'D'")
   .whenNotMatchedInsert(
       values={"cust_id": "s.cust_id", "name": "s.name",
               "created_at": "current_timestamp()"}
   )
   .execute())

# Maintenance & Time Travel
dt.optimize().executeCompaction()
dt.vacuum(168)  # 7 days
hist_df = dt.history()
```

Auto Loader & Structured Streaming

Streaming

```python
# Ingestion with Databricks Auto Loader (cloudFiles)
stream_df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/checkpoints/schema")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("abfss://raw@adls.dfs.core.windows.net/events/"))

# Watermarked Sink write with AvailableNow trigger
query = (stream_df
    .withWatermark("event_time", "10 minutes")
    .writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/checkpoints/bronze_events")
    .trigger(availableNow=True) # Incremental batch
    .toTable("bronze.events"))
```

Trigger(availableNow=True): Replaces legacy `once=True`. Processes all pending data in rate-limited micro-batches then stops, slashing cluster costs!

PyArrow Vectorized Pandas UDFs

Performance

```python
import pandas as pd
from pyspark.sql.functions import pandas_udf

# Vectorized Series-to-Series UDF (Uses Apache Arrow)
@pandas_udf(T.DoubleType())
def calculate_vat(price: pd.Series, rate: pd.Series) -> pd.Series:
    return price * (1.0 + rate)

df_taxed = df.withColumn("gross", calculate_vat(F.col("net"), F.col("rate")))

# Grouped Map Vectorized UDF (applyInPandas)
def normalize_group(pdf: pd.DataFrame) -> pd.DataFrame:
    pdf["z_score"] = (pdf["val"] - pdf["val"].mean()) / pdf["val"].std()
    return pdf

df_norm = df.groupby("device_id").applyInPandas(
    normalize_group, schema=output_schema
)
```

Avoid Standard @udf: Regular Python UDFs force row-by-row Py4J socket serialization. Pandas UDFs process columnar Arrow batches up to 100x faster!

Partition Management: Repartition vs Coalesce

FinOps / Sizing

```python
# Inspect current partitions
num_parts = df.rdd.getNumPartitions()

# Repartition: FULL SHUFFLE across cluster (Increase/Balance)
df_balanced = df.repartition(200)
df_hash = df.repartition(100, "region")

# Coalesce: ZERO SHUFFLE (Shrinks local partition blocks)
df_shrunk = df.coalesce(10) # Use right before file write!
```

Operation	Network Shuffle	Partition Target
`repartition(N)`	Yes (Exchange)	Increase or redistribute evenly
`coalesce(N)`	No (Local merge)	Shrink only; avoids small files

Rule of Thumb: Target **100MB - 200MB** uncompressed data per partition.

Caching, Persistence Levels & Cleanup

Memory Management

```python
from pyspark import StorageLevel

# Cache: MEMORY_AND_DISK_DESER (Default)
df_cached = df.cache()

# Persist with Serialized Kryo (Saves up to 3x RAM)
df_persisted = df.persist(StorageLevel.MEMORY_ONLY_SER)

# MANDATORY: Evict from RAM when actions complete!
df_cached.unpersist()
```

- **When to Cache:** Cache ONLY when a DataFrame is reused across 2+ separate downstream actions.
- **Over-caching Antipattern:** Starves Unified Execution Memory, forcing shuffle spills to disk!

Streaming foreachBatch Microbatch Processing

Streaming Sink

```python
def process_micro_batch(micro_df, batch_id):
    # Cache microbatch if referenced in multiple merges
    micro_df.persist()

    # ACID Merge microbatch directly into Delta Silver
    (DeltaTable.forName(spark, "silver.txns").alias("t")
     .merge(micro_df.alias("s"), "t.id = s.id")
     .whenMatchedUpdateAll()
     .whenNotMatchedInsertAll()
     .execute())

    micro_df.unpersist()

query = (stream_df.writeStream
    .foreachBatch(process_micro_batch)
    .option("checkpointLocation", "/chk/silver")
    .start())
```

foreachBatch Superpower: Allows writing a streaming pipeline into non-streaming sinks, running multi-table merges, and executing deduplications per microbatch.

SPARK ARCHITECTURE & ENGINE INTERNALS

Cluster Topology, JVM Memory Allocation, Shuffling, Catalyst & Resource Sizing

SPARK CORE | TUNGSTEN & PHOTON | MEMORY MODEL | CATALYST OPTIMIZER

Cluster Topology & Execution Hierarchy

Architecture

Component	Role & Responsibilities
Driver	Hosts `SparkSession`, DAGScheduler, TaskScheduler, Web UI (4040). Translates code to physical stages.
Cluster Mgr	Allocates physical compute resources (Databricks CM / YARN / K8s).
Worker Node	Physical VM hosting Executor JVM instances and NVMe shuffle disks.
Executor	JVM process running tasks in thread slots; holds execution/storage RAM.
Core / Slot	Thread processing slot. Optimal: **4-5 cores per executor**.

Execution Cascade: **Action** (collect/write) → **Job** → **Stages** (split by wide shuffles) → **Tasks** (1 task per partition per core).

Executor JVM Unified Memory Model

Memory

Segment	Size / Share	Purpose
Reserved	300 MB	Hardcoded for Spark internal engine structures
User Memory	40% (Heap - 300MB)	Custom data structures, UDF allocations, metadata
Unified Memory	60% (Heap - 300MB)	Dynamically shared between Execution & Storage
└ *Execution*	Dynamic pool	Shuffles, joins, sorts, aggregations (Can evict storage!)
└ *Storage*	Dynamic pool	Cached data (`df.cache()`); cannot evict execution!
Overhead	max(384MB, 10%)	Off-heap VM overhead, Py4J IPC, thread stacks

Off-Heap (Tungsten): `spark.memory.offHeap.enabled = true` allocates memory directly outside JVM, completely bypassing Garbage Collection pauses!

Transformations: Narrow vs Wide Dependencies

DAG Scheduler

Narrow (Pipelined)

- `map`, `filter`, `withColumn`, `union`
- Each parent partition used by at most 1 child partition.
- Zero network shuffle! Pipelined in memory within the **same stage**.

Wide (Shuffle Boundary)

- `groupBy`, `join`, `distinct`, `repartition`
- Multiple child partitions depend on parent partitions.
- Forces network exchange; terminates stage and starts new stage!

The Shuffle Engine: Write, Read & Spill

Shuffle

- **Shuffle Write:** Mappers write records to memory buffer; sorted and written to a single `.data` file with `.index` offset file on local NVMe disk.
- **Shuffle Read:** Reducers fetch their required blocks over Netty RPC from all mappers across the network into local memory buffers.
- **Spill (Memory) vs Spill (Disk):** Occurs when shuffle memory exceeds executor threshold. Data is serialized and flushed to disk (huge latency penalty!).

Triage Spills: If Spark UI shows Spill (Memory) / Spill (Disk), immediately increase executor RAM, enable AQE, or increase shuffle partitions!

Catalyst Optimizer & Tungsten Engine

Query Engine

1. Analysis
Resolves names against Unity Catalog.

2. Logical Opt
Predicate pushdown, column pruning.

3. Physical Plan
CBO picks optimal join (BHJ vs SMJ).

4. Tungsten CodeGen
Compiles loops into CPU L1/L2 registers.

UnsafeRow: Tungsten stores records in raw off-heap binary format without Java object wrapper overhead.

Cluster Sizing & Resource Allocation Math

Infrastructure

```
# Production Cluster Allocation Formula
# Example: 10 Nodes, 16 Cores & 64GB RAM each
1. Cores per Node = 16 - 1 (OS/YARN) = 15 Cores
2. Cores per Executor = 5 (Sweet spot for HDFS/ADLS IO throughput)
3. Executors per Node = 15 / 5 = 3 Executors
4. Total Executors = (10 Nodes * 3) - 1 (Driver) = 29 Executors
5. RAM per Node = 64GB - 1GB (OS) = 63GB
6. RAM per Executor = 63GB / 3 = 21GB
7. Memory Overhead = max(384MB, 0.10 * 21GB) = ~2.1GB
8. Executor Heap = 21GB - 2.1GB = ~18.9GB (Set: --executor-memory 19g)
```

Parallelism Rule: `spark.default.parallelism = total_executor_cores * 2 to 3` (For 145 cores = ~300-450 partitions).

SPARK ARCHITECTURE & ENGINE INTERNALS

Photon Native Engine, AQE Dynamic Optimization, Spark UI Triage & Skew Mastery

AQE & SKEW | SPARK UI DEBUGGING | PHOTON C++

Databricks Photon Native Vectorized Engine

Photon

- **C++ Native Execution:** Replaces Tungsten Java bytecode execution path entirely with hand-optimized native C++.
- **SIMD Vectorization:** Processes batches of column values in CPU SIMD registers simultaneously (Single Instruction, Multiple Data).
- **Zero JVM Overhead:** Eliminates Java object allocation, GC pauses, and JIT deoptimizations for high-concurrency SQL and aggregations.

Accelerated Operations: Delta Parquet scanning, complex hash joins, high-cardinality aggregations, and string/regex operations execute up to **5x faster** on Photon.

Adaptive Query Execution (AQE) 3 Superpowers

Adaptive Tuning

Feature	Runtime Action	Config Knob
Dynamic Partition Coalescing	Merges tiny post-shuffle partitions to eliminate small-task scheduling overhead	`spark.sql.adaptive.coalescePartitions.enabled = true`
Dynamic Join Switching	Converts SortMergeJoin to BroadcastHashJoin if runtime stage output < 10MB	`spark.sql.adaptive.autoBroadcastJoinThreshold`
Dynamic Skew Join	Detects skewed partitions (>5x median & >256MB) and automatically splits them	`spark.sql.adaptive.skewJoin.enabled = true`

Spark UI Diagnostic & Triage Playbook

Diagnostics

Spark UI Tab	Metric / Symptom	Diagnosis & Fix
Stages Tab	Max task time >> 75th percentile	**Data Skew:** Single straggler task holding stage. Enable AQE skew join or salt keys.
Stages Tab	Spill (Memory) & Spill (Disk) > 0	**Memory Deficit:** Increase executor memory or raise `spark.sql.shuffle.partitions`.
Executors Tab	GC Time > 10% of Task Time	**Heap Exhaustion:** Large object allocations or oversized partitions. Enable Off-Heap memory.
SQL / Plan	SortMergeJoin instead of Broadcast	Check table statistics with `ANALYZE TABLE` or supply `/*+ BROADCAST */` hint.

Data Skew Root Causes & Solutions

Skew Remediation

- **Root Causes:** High frequency keys (e.g. `NULL` customer IDs, default dummy values, power-law Pareto distributions).
- **Strategy 1 (AQE):** Ensure `spark.sql.adaptive.skewJoin.enabled = true` (Databricks default).
- **Strategy 2 (Null Separation):** Filter null keys into a separate stream; join only non-nulls; `UNION ALL` nulls back.
- **Strategy 3 (Broadcast):** If one table fits in RAM (<8GB on large clusters), broadcast it to eliminate shuffle completely!

Storage Levels & Checkpoint Decision Matrix

Persistence

Storage Level	Space	CPU	RAM	Disk
`MEMORY_ONLY`	High	Lowest	Yes (Deser)	No
`MEMORY_ONLY_SER`	Low (3x less)	Low	Yes (Kryo)	No
`MEMORY_AND_DISK`	Medium	Med	Spills to Disk	Yes
`OFF_HEAP`	Low	Lowest	Tungsten RAM	No GC!

df.persist() vs df.checkpoint(): `persist()` keeps lineage for node recovery; `checkpoint()` writes to ADLS and **TRUNCATES lineage** graph (mandatory for iterative ML/Streaming).

JVM Garbage Collection (G1GC) & Shuffle Math

Tuning Knobs

```
# Production G1GC Flags for Databricks / Spark
-XX:+UseG1GC
-XX:InitiatingHeapOccupancyPercent=35
-XX:G1ReservePercent=15
-XX:MaxGCPauseMillis=200

# Optimal Shuffle Partitions Calculation
# Rule: Target 100MB - 200MB per shuffle partition
Total Shuffle Input Data = 64 GB (= 65,536 MB)
Optimal Partitions = 65,536 MB / 128 MB = 512 Partitions
Set: spark.conf.set("spark.sql.shuffle.partitions", 512)
```

AQE Advisory Size: Set `spark.sql.adaptive.advisoryPartitionSizeInBytes = 134217728` (128MB) for automatic post-shuffle partition sizing.

PYTHON IN DATABRICKS CHEAT SHEET

Production Reference: DBUtils, Delta Live Tables (DLT), pandas on Spark, SDK & Magic Commands

PYTHON 3.10+ | DBUTILS | DELTA LIVE TABLES | DATABRICKS SDK

Databricks Utilities (dbutils) Master Reference

dbutils

```python
# 1. File System (dbutils.fs) & Volumes
dbutils.fs.ls("/Volumes/prod/finance/raw_files/")
dbutils.fs.cp("abfss://landing@adls.dfs.core.windows.net/data.csv",
              "/Volumes/prod/finance/raw_files/", recurse=True)
dbutils.fs.rm("/tmp/scratch/", recurse=True)
dbutils.fs.put("/tmp/config.json", '{"env":"prod"}', overwrite=True)

# 2. Secrets & Azure Key Vault
secret_token = dbutils.secrets.get(scope="akv-scope", key="storage-key")
scopes = dbutils.secrets.listScopes()

# 3. Notebook Workflows
res = dbutils.notebook.run("sub_pipeline", timeout_seconds=1800,
                           arguments={"batch_id": "101"})
dbutils.notebook.exit("SUCCESS: 154000 records processed")

# 4. Interactive Widgets
dbutils.widgets.text("env", "dev", "Target Environment")
env_val = dbutils.widgets.get("env")
```

Delta Live Tables (DLT) Declarative Pipelines

DLT

```python
import dlt
from pyspark.sql import functions as F

# Streaming Ingestion Table
@dlt.table(
    name="bronze_txns",
    comment="Raw transaction stream",
    table_properties={"quality": "bronze"}
)
def bronze_txns():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "json")
            .load("abfss://landing@adls.dfs.core.windows.net/txns/"))

# Silver Table with Data Quality Expectations
@dlt.table(comment="Validated transactions")
@dlt.expect_or_drop("valid_id", "txn_id IS NOT NULL")
@dlt.expect_or_fail("valid_amt", "amount > 0.0")
def silver_txns():
    return dlt.read_stream("bronze_txns").filter(F.col("status") != "REJECT")
```

DLT Expectations: `expect` (logs metric), `expect_or_drop` (drops bad rows), `expect_or_fail` (fails pipeline on violation).

CDC Ingestion with dlt.apply_changes()

SCD Type 1 & 2

```python
import dlt

# Automatic SCD Type 2 History Tracking
dlt.create_streaming_table("gold_customers_scd2")

dlt.apply_changes(
    target="gold_customers_scd2",
    source="silver_customer_cdc",
    keys=["customer_id"],
    sequence_by="updated_timestamp",
    apply_as_deletes="operation = 'DELETE'",
    except_column_list=["operation"],
    stored_as_scd_type=2 # Auto creates __START_AT and __END_AT
)
```

- `stored_as_scd_type=1` overwrites records in place (latest snapshot).
- `stored_as_scd_type=2` manages effective date ranges and active flags out-of-the-box!

pandas API on Spark (pyspark.pandas)

Scale pandas

```python
import pyspark.pandas as ps

# Read distributed data using familiar pandas syntax
psdf = ps.read_parquet("abfss://gold@adls.../sales/")

# Distributed GroupBy & Summary Aggregations
summary = psdf.groupby("country")["revenue"].agg(["mean", "sum", "count"])

# Seamless Two-Way Conversion
spark_df = psdf.to_spark()
ps_df_again = spark_df.to_pandas_on_spark()
```

Anti-Pattern: Avoid calling `.to_pandas()` or `.iloc` on large DataFrames as it pulls all distributed data into driver RAM, risking OOM!

Databricks Connect v2 & Databricks Python SDK

Developer Tooling

```python
# Databricks Connect v2 (Local IDE to Remote Cluster over gRPC)
from databricks.connect import DatabricksSession
spark = DatabricksSession.builder.getOrCreate()
df = spark.read.table("prod_catalog.silver.transactions")
df.filter("amount > 1000").show()

# Databricks Python SDK (Workspace Automation)
from databricks.sdk import WorkspaceClient
w = WorkspaceClient()
run = w.jobs.run_now(job_id=892145)
clusters = w.clusters.list()
```

Thin Client: Databricks Connect v2 sends unresolved DataFrame plans via gRPC; execution runs entirely on the remote Databricks cluster.

Databricks Magic Commands & Scoping

Notebook Magic

Magic	Language / Action	Behavior & Scope
`%python`	Python 3	Default language; shares local variables across notebook cells
`%sql`	Spark SQL	Executes SQL query; accessible via `_sqldf` DataFrame
`%sh`	Bash Shell	Runs bash on the driver node container
`%fs`	Databricks FS	Shorthand for `dbutils.fs` (e.g. `%fs ls /Volumes`)
`%pip`	pip install	Installs library scoped to the current notebook session only
`%run`	Sub-notebook	Executes another notebook in-line and inherits all its variables

%run vs dbutils.notebook.run: `%run` shares memory & variables in the same session; `dbutils.notebook.run` executes in an isolated ephemeral job context.

🐍 PYTHON IN DATABRICKS CHEAT SHEET

Wheels, Databricks Asset Bundles (DABs), Observability, Resilience, Types & Unit Testing

DABS / CI/CD | PRODUCTION WHEELS | OBSERVABILITY

📦 Wheels & Unity Catalog Volumes

Packaging

```python
# In-Notebook ephemeral dependency install
%pip install pydantic==2.8.2 tenacity==8.4.1

# Production Wheel Distribution via Unity Catalog Volumes
# Store built wheel in governed volume:
# /Volumes/prod_catalog/core_libs/my_etl-1.2.0-py3-none-any.whl
%pip install /Volumes/prod_catalog/core_libs/my_etl-1.2.0-py3-none-any.whl

# Import modular reusable package in notebook
from my_etl.transformers import BankingNormalizer
normalizer = BankingNormalizer(currency="EUR")
```

Volumes for Libraries: Replacing legacy DBFS root with Unity Catalog Volumes ensures strict access control and auditing for third-party binary artifacts.

🚀 Databricks Asset Bundles (DABs) CI/CD

DevOps

```yaml
# databricks.yml definition snippet
bundle:
  name: bfsi_settlement_pipeline

targets:
  dev:
    workspace:
      host: https://adb-dev.azuredatabricks.net
  prod:
    workspace:
      host: https://adb-prod.azuredatabricks.net

resources:
  jobs:
    nightly_settlement:
      name: "[prod] Nightly Settlement Job"
      tasks:
        - task_key: run_dlt
          pipeline_task:
            pipeline_id: 489a-bc12-dlt-id
```

CLI Command	Action
`databricks bundle validate`	Lints and verifies bundle syntax
`databricks bundle deploy -t prod`	Deploys resources to target workspace
`databricks bundle run nightly_settlement`	Executes workflow immediately

📊 Structured Logging & Observability

Observability

```python
import json, logging
from datetime import datetime

class StructuredPipelineLogger:
    def __init__(self, pipeline_name: str):
        self.name = pipeline_name

    def log_event(self, event_name: str, records: int, duration_sec: float):
        payload = {
            "timestamp": datetime.utcnow().isoformat(),
            "pipeline": self.name,
            "event": event_name,
            "records_processed": records,
            "duration_sec": duration_sec
        }
        print(json.dumps(payload)) # Ingested into Azure Log Analytics

audit = StructuredPipelineLogger("SEPA_Payment_Processor")
audit.log_event("STAGE_COMPLETED", records=845000, duration_sec=14.2)
```

🔄 Concurrent Extraction & Exponential Retries

Resilience

```python
from concurrent.futures import ThreadPoolExecutor
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=2, max=10))
def fetch_api_page(page_id: int) -> dict:
    # Resilient REST API caller
    return {"page": page_id, "data": [1, 2, 3]}

# Parallelize lightweight API calls on Driver before Spark processing
with ThreadPoolExecutor(max_workers=8) as executor:
    results = list(executor.map(fetch_api_page, range(1, 50)))
```

Thread Concurrency: Use Python `ThreadPoolExecutor` on the driver for I/O-bound REST calls, then parallelize the dataset with `spark.createDataFrame()`.

🛡️ Type Annotations & Schema Contracts

Data Quality

```python
from pyspark.sql import DataFrame, Column
from typing import Callable

def apply_curation(
    df: DataFrame,
    primary_key: str,
    business_rule: Callable[[Column], Column]
) -> DataFrame:
    assert primary_key in df.columns, f"Missing required key: {primary_key}"
    return df.filter(business_rule(df[primary_key]))
```

- Enforce PEP 484 type hints across modular libraries.
- Pre-flight assertions catch schema mismatch errors in milliseconds before distributed stages start.

🧪 Unit Testing PySpark Pipelines in CI/CD

Testing

```python
import pytest
from pyspark.testing.utils import assertDataFrameEqual

def test_filter_active(spark):
    input_data = [(1, "ACTIVE"), (2, "CANCELLED")]
    input_df = spark.createDataFrame(input_data, ["id", "status"])

    expected_data = [(1, "ACTIVE")]
    expected_df = spark.createDataFrame(expected_data, ["id", "status"])

    result_df = input_df.filter("status = 'ACTIVE'")
    # Spark 3.5+ built-in test assertion
    assertDataFrameEqual(result_df, expected_df)
```

pyspark.testing: Use `assertDataFrameEqual` and `assertSchemaEqual` in pytest to validate transformations without third-party libraries.